

Electricity Demand Forecasting

Project Theory & Complete Explanation (XGBoost Model)

1. Problem Statement

Electricity grid operators need to know in advance how much power will be needed in the next hour, day, or week. Electricity cannot be stored at scale, so it must be generated in near real-time to match demand. Under-forecasting leads to blackouts or load-shedding, while over-forecasting leads to wasted generation and higher operating cost. Accurate demand forecasting is therefore critical for grid planning, and is used daily by utility companies and grid operators worldwide.

The goal of this project: using 5 years of hourly historical data, build a model that predicts electricity demand for future time periods.

2. Understanding the Data

Column	Meaning
Timestamp	Hourly record of date and time
Temperature, Humidity	Weather conditions - directly affect demand (AC/heater usage)
Demand	Target variable to be predicted (electricity load)
hour, dayofweek, month, year, dayofyear	Calendar breakdown of the timestamp

Electricity demand is fundamentally a **time-series** problem, and it typically shows three kinds of patterns:

- **Trend** - overall demand tends to rise year over year (population growth, industrial activity).
- **Seasonality** - daily patterns (low at night, high during the day), weekly patterns (weekday vs weekend), and yearly patterns (summer vs winter).
- **Weather dependency** - demand spikes during extreme temperatures due to cooling/heating equipment usage.

3. Data Cleaning - Why Different Methods Were Used

Missing values were handled differently depending on the nature of each column, because one strategy does not fit all column types:

- **Forward fill (ffill)** on calendar columns (hour, month, etc.) - since these form a continuous, orderly sequence, carrying the previous row's value forward is reasonable.
- **Backward fill (bfill)** on Temperature/Humidity - weather changes gradually, so filling with a nearby known value is a reasonable approximation.
- **Time-based interpolation** on Demand - this estimates missing values proportionally to elapsed time between two known points, which suits a continuously changing quantity like demand far better than a simple mean fill (which would ignore the underlying trend).

4. Feature Engineering - The Most Important Part

a) Calendar Features (quarter, weekofyear, is_weekend)

Demand patterns are cyclical - Monday differs from Sunday, December differs from June. These features tell the model what point in the cycle it is looking at, so it can learn seasonal and weekly patterns.

b) Lag Features - the single most powerful features in this model

Demand_lag_24hr = demand at the same hour, one day ago. **demand_lag_168hr** = demand at the same hour, one week ago ($24 \times 7 = 168$ hours).

Electricity demand shows strong **autocorrelation** - meaning demand today tends to resemble demand at the same hour yesterday, and especially the same hour on the same weekday last week. This is a classic time-series technique known as a **"naive seasonal" / persistence signal**. Giving the model these two concrete reference points (yesterday, last week) gives it an extremely strong starting point for prediction - past demand is, in effect, the best predictor of future demand.

In this project's feature-importance results, Demand_lag_24hr and demand_lag_168hr together accounted for roughly 93% of the model's total decision-making weight - which explains why the final model achieved an R-squared of about 0.985.

c) Rolling Features (24-hour rolling mean and standard deviation)

Rolling mean captures the recent short-term average demand level (is overall demand currently trending high or low). **Rolling standard deviation** captures how volatile or stable demand has been recently. These are "smoothed" features that reduce noise compared to using a single past data point.

5. Train-Test Split - Why Date-Based, Not Random

The data was split chronologically: training data up to 31 Dec 2023, and test data from 1 Jan 2024 onward - a random split was deliberately avoided.

Why this matters: In time-series problems, future data must never leak into the training set. A random split could place a future data point into training and a past point into testing, giving the model indirect access to information it would not have in real-world deployment. This produces unrealistically high accuracy during evaluation, but the model would fail in production. Chronological splitting mirrors how the model will actually be used: only past data is available to predict the future.

6. Model - Why XGBoost

XGBoost (Extreme Gradient Boosting) builds many small decision trees sequentially. Each new tree focuses on correcting the errors (residuals) made by the previous trees. The final prediction is a weighted combination of all the trees.

Reasons for choosing XGBoost for this problem:

- Handles **non-linear relationships** well - for example, both very high and very low temperatures increase demand, which is not a straight-line relationship.

- Performs strongly on **tabular/structured data** - consistently a top performer in this data type across benchmarks and competitions.
- Naturally handles mixed feature types and is relatively robust to unscaled inputs.
- Trains efficiently even on large datasets.

Key hyperparameters used

Parameter	Meaning
n_estimators = 1000	Maximum number of trees the model can build
learning_rate = 0.01	Each tree contributes a small amount - slow, careful learning that reduces overfitting
early_stopping_rounds = 50	Stop training if test performance does not improve for 50 rounds - prevents overfitting
objective = 'reg:squarederror'	This is a regression task; the model minimizes squared error

7. Evaluation Metrics

- **RMSE (Root Mean Squared Error)** - squares the errors, averages them, then takes the square root. Because errors are squared, larger errors are penalized disproportionately more.
- **MAE (Mean Absolute Error)** - the average of absolute errors; every error is weighted equally regardless of size.
- **R-squared (R2)** - the proportion of variance in demand that the model explains, on a scale of 0 to 1. Closer to 1 is better.

Results achieved on this dataset: **RMSE ~174.8**, **MAE ~123.3** (about 2.4% of mean demand), **R2 ~0.985**. RMSE being noticeably higher than MAE indicates a small number of larger, outlier-type errors exist, but the overall error rate is low.

8. Full Pipeline Summary

Raw Data → Clean missing values → Feature engineering (lag, rolling, calendar) → Chronological train/test split → Train XGBoost → Evaluate (RMSE/MAE/R2) → Save trained model → Deploy

Why this shapes the deployment approach

Because the model depends heavily on lag features (recent actual demand), it is best suited for **day-ahead / batch forecasting** rather than zero-history real-time prediction. In practice, this becomes a scheduled job: each day, the system pulls the most recent demand history, builds the lag/rolling features from it, and forecasts the next day's demand - which mirrors exactly how real grid operators plan generation one day in advance.